优化算法

前缀和

定义

前缀和是一种重要的预处理，能大大降低查询的时间复杂度。可以简单理解为“数列的前n项的和”。

假设数组元素$a[1]=1,\ a[2]=2,\ a[3]=3,\ \ldots,\ a[n]=n$

那么数组的前缀和为：

$s[1]=a[1]$

$s[2]=a[1]+a[2]$ 即：$s[2]=s[1]+a[2]$

$s[3]=a[1]+a[2]+a[3]$ 即：$s[3]=s[2]+a[3]$

……

$s[n]=a[1]+a[2]+a[3]+\ldots+a[n]$ 即：$s[n]=s[n-1]+a[n]$

U53525 前缀和（例题）

算法分析：这是一道前缀和的裸题。我们来看下前缀和的代码

代码参考：

```
#include<bits/stdc++.h>
using namespace std;
const int N=110;
int a[N];
int s[N];
int n;
int main() {
    cin>>n;
    for(int i=1;i<=n;i++)   cin>>a[i];
    for(int i=1;i<=n;i++)   s[i]=s[i-1]+a[i];   //前缀和计算公式
    for(int i=1;i<=n;i++)   cout<<s[i]<<" ";
    return 0;
}
```

作用

假设要求解区间$[3,6]$的总和。我们最简单的做法就是把区间的元素遍历一遍进行累加。但是其时间复杂度是$O(n)$的。

前缀和则是在$O(1)$ 的时间复杂度上，求解区间和。

假设求$s[2,5]$的区间和。具体思路为：

1. 先预处理前缀和$s[1]$，$s[2]$，$s[3]$，$\ldots$，$s[n]$。
2. $s[2,5]=a[2]+a[3]+a[4]+a[5]$。 $s[5]=a[1]+a[2]+a[3]+a[4]+a[5]$。 $s[1]=a[1]$。 从这3个式子，我们可以推出公式：$s[2,5]=s[5]-s[1]$。即$s[L,R]=s[R]-s[L-1]$。

3. 通过这个公式，我们就可以在$O(1)$的时间复杂度上完成区间和的求解了。

P5745 【深基附B例】数列求和

算法分析：

- 任务1：$n \leq 200$

最直接的思路就是枚举所有子区间的头尾i和j，然后对i和j里面的所有数字累加起来求和，再判断是否在不大于M的情况下最大。

```
#include<bits/stdc++.h>
using namespace std;
const int N=4000010;

int a[N];
int n,m;
int ans,ansi,ansj;
int main() {
    cin>>n>>m;
    for(int i=1;i<=n;i++)   cin>>a[i];
    for(int i=1;i<=n;i++) {
        for(int j=i;j<=n;j++) {
            int sum=0;
            for(int k=i;k<=j;k++) sum+=a[k];    //计算区间[i,j]的总和
            if(sum<=m && sum>ans){
                ansi=i; ansj=j; ans=sum;
            }
        }
    }
    cout<<ansi<<" "<<ansj<<" "<<ans;
    return 0;
}
```

算法的时间复杂度是$O(n^3)$

- 任务2：$n \leq 3000$

枚举每次区间，计算区间和是很浪费时间的。这里我们可以采用前缀和将第三层循环优化到$O(1)$。算法的时间复杂度是$O(n^2)$。

```
#include<bits/stdc++.h>
using namespace std;
const int N=4000010;

int a[N];
long long s[N];
int n,m;
int ans,ansi,ansj;
int main() {
    cin>>n>>m;
    for(int i=1;i<=n;i++)   cin>>a[i];
    for(int i=1;i<=n;i++)   s[i]=s[i-1]+a[i];

    for(int i=1;i<=n;i++) {
        for(int j=i;j<=n;j++) {
            int sum=s[j]-s[i-1];
            if(sum>m)   break;      //优化小常数
```

```
            if(sum<=m && sum>ans){
                ansi=i; ansj=j; ans=sum;
            }
        }
    }
    cout<<ansi<<" "<<ansj<<" "<<ans;
    return 0;
}
```

- 任务3：$n \leq 10^5$

我们在枚举所有的i，对于给定的i，要求$s[j]-s[i-1] \leq m$。这就需要找到最大的j。而我们发现，前缀和数组s是存在一定单调性。那么就可以采用二分的方式，花费$O(log\ n)$的时间找到j。总的时间复杂度是$O(nlog\ n)$。

```
#include<bits/stdc++.h>
using namespace std;
const int N=4000010;

int a[N];
long long s[N];
int n,m;
int ans,ansi,ansj;

int main() {
    cin>>n>>m;
    for(int i=1;i<=n;i++)   cin>>a[i];
    for(int i=1;i<=n;i++)   s[i]=s[i-1]+a[i];

    for(int i=1;i<=n;i++) {
        int x=s[i-1]+m+1;
        int j=lower_bound(s+1,s+1+n,x) - s-1;   //查找大于等于x的值
        int sum=s[j]-s[i-1];

        if(sum<=m && sum>ans){
            ansi=i; ansj=j; ans=sum;
        }
    }
    cout<<ansi<<" "<<ansj<<" "<<ans;
    return 0;
}
```

- 任务4：$n \leq 4 \times 10^6$

然而，本题要求算法是$O(n)$，这里我们需要找到一个更优更快的方法。这里我们来看一种比较巧妙的算法。

尺取法（双指针法）。

我们创建两个指针Li，Ri。 如果$s[Ri]-s[Li-1]>m$，则把Li指针向右移动。

否则每次循环就移动右指针Ri。这指针移动的过程中更新区间和最终解。

```
#include<bits/stdc++.h>
using namespace std;
const int N=4000010;

long long s[N];
```

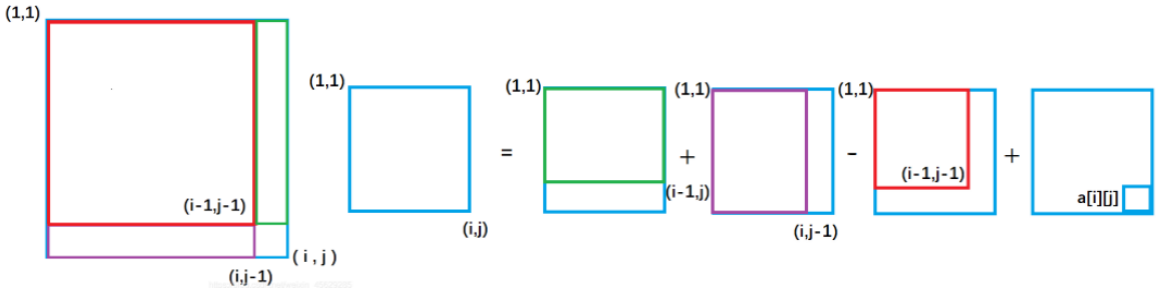
```cpp
int n,m;
int Le,Ri,ans;
int La,Ra;
int main() {
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        int x;  cin>>x;
        s[i]=s[i-1]+x;       //计算前缀和
    }
    Le=1;
    while(Ri<=n) {
        while(s[Ri]-s[Le-1]>m && Ri>Le) Le++;    //当左右指针还没重叠且区间和溢出时，移动左指针
        if(s[Ri]-s[Le-1]>ans) {      //最优解可以更新
            La=Le;  Ra=Ri;  ans=s[Ri]-s[Le-1];
        }
        Ri++;        //每次都移动右指针
    }
    cout<<La<<" "<<Ra<<" "<<ans;
    return 0;
}
```

相应题型推荐：

- P6568 [NOI Online #3 提高组] 水壶

【扩展】二维前缀和

同一维前缀和一样，我们先来定义一个二维数组$s[][]$, $s[i][j]$表示二维数组中，左上角$(1,1)$到右下角(i,j)所包围的矩阵元素的和。接下来推导二维前缀和的公式。

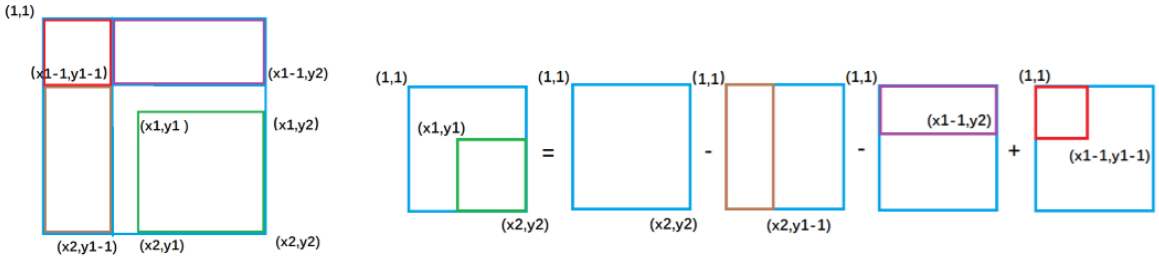


图中我们很容易看出:

整个外围蓝色矩形面积$s[i][j]$= 绿色面积$s[i-1][j]$ + 紫色面积$s[i][j-1]$ - 重复加的红色的面积$s[i-1][j-1]$+小方块的面积$a[i][j]$;

因此得出二维前缀和预处理公式$s[i][j] = s[i-1][j] + s[i][j-1] + a[i][j] - s[i-1][j-1]$

接下来回归问题去求以$(x1,y1)$为左上角和以$(x2,y2)$为右下角的矩阵的元素的和。



绿色矩形的面积 = 整个外围面积$s[x2,y2]$ - 黄色面积$s[x2][y1-1]$- 紫色面积$s[x1-1][y2]$ + 重复减去的红色面积 $s[x1-1][y1-1]$

因此二维前缀和的结论为：

以$(x1,y1)$为左上角，$(x2,y2)$为右下角的子矩阵的和为：
$s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1]$

P1719 最大加权矩形

算法分析：

这是一道二维前缀和的裸题。我们先求解初始化的二维前缀和$s[i][j]$。然后在逐一枚举每一个子矩阵，求解最优解即可。

代码参考：

```
#include<bits/stdc++.h>
using namespace std;
const int N=125;
int a[N][N];
int s[N][N];
int ans=-1e9;   //初始化负无穷
int n;
int main() {
    cin>>n;
    for(int i=1;i<=n;i++) {
        for(int j=1;j<=n;j++)   cin>>a[i][j];
    }
    for(int i=1;i<=n;i++) {
        for(int j=1;j<=n;j++) {
            s[i][j]=s[i-1][j]+s[i][j-1]+a[i][j]-s[i-1][j-1];    //初始化二维前缀和
        }
    }
    for(int x1=1;x1<=n;x1++) {      //逐一枚举起点(x1,y1)和终点(x2,y2)
        for(int y1=1;y1<=n;y1++) {
            for(int x2=x1;x2<=n;x2++) {
                for(int y2=y1;y2<=n;y2++) {
                    int x=s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1];  //计算当前子矩阵
                    ans=max(ans,x);
                }
            }
        }
    }
    cout<<ans;
    return 0;
}
```

题型推荐：

- P2004 领地选择
- P3406 海底高铁
- P2280 [HNOI2003]激光炸弹

差分

定义

类似于数学中的求导和积分，**差分可以看成前缀和的逆运算。**

差分数组：

首先给定一个原数组a：$a[1],a[2],a[3],\ldots,a[n]$;

然后我们构造一个数组b：$b[1],b[2],b[3],\ldots,b[i]$;

使得 $a[i]=b[1]+b[2]+b[3]+,\ldots,+b[i]$

也就是说，a数组是b数组的前缀和数组，反过来我们把b数组叫做a数组的差分数组。

换句话说，每一个$a[i]$都是b数组中从头开始的一段区间和。

构造的方式也比较简单，我们来看下思路：

`a[0]= 0;` `b[1] = a[1] - a[0];` `b[2] = a[2] - a[1];` `b[3] =a [3] - a[2];`

`........`

`b[n] = a[n] - a[n-1];`

U69096 前缀和的逆

代码实现

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N=110;
int a[N],b[N];

int n;
int main() {
    cin>>n;
    for(int i=1;i<=n;i++)   cin>>a[i];
    for(int i=1;i<=n;i++)   b[i]=a[i]-a[i-1];
    for(int i=1;i<=n;i++)   cout<<b[i]<<" ";
    return 0;
}
```

作用

给定区间$[l,r]$，让我们把a数组中的$[l,r]$区间中的每一个数都加上c,即
$a[l]+c,\ a[l+1]+c,\ a[l+2]+c\ ,\ldots,\ a[r]+c$;

暴力做法是for循环l到r区间，时间复杂度$O(n)$，如果我们需要对原数组执行m次这样的操作，时间复杂度就会变成$O(nm)$。有没有更高效的做法吗? 考虑差分做法。

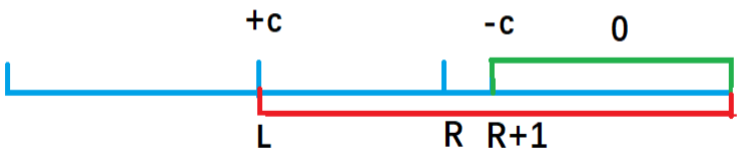
我们定义a数组是b数组的前缀和数组，比如对b数组的$b[i]$的修改，会影响到a数组中**从$a[i]$及往后的每一个数。**

首先让差分b数组中的 $b[l]+c$,a数组变成$a[l]+c,\ \ a[l+1]+c,\ \ \ldots,\ \ a[n]+c$;

然后我们打个补丁，$b[r+1]-c$, a数组变成 $a[r+1]-c,\ \ a[r+2]-c,\ \ \ldots,\ \ a[n]-c$;

为啥还要打个补丁？

我们画个图理解一下这个公式的由来:



$b[l]+c$，效果使得a数组中$a[l]$及以后的数都加上了c(红色部分)，但我们只要求$[l,r]$区间加上c, 因此还需要执行$b[r+1]-c$,让a数组中$a[r+1]$及往后的区间再减去c(绿色部分)，这样对于$a[r]$以后区间的数相当于没有发生改变。

因此我们得出一维差分结论：给a数组中的$[l,r]$区间中的每一个数都加上c,只需对差分数组b做$b[l]+=c, b[r+1]-=c$。时间复杂度为$O(1)$, 大大提高了效率。

P2367 语文成绩

算法分析：通过题意分析，可以得到这是一道一维差分的裸题。题意也把一些限制说清楚了

代码参考：

```
#include<bits/stdc++.h>
using namespace std;
const int N=5000010;
int a[N],b[N];
int n,p;

int main() {
    scanf("%d%d",&n,&p);
    for(int i=1;i<=n;i++)   scanf("%d",&a[i]);
    //创建差分数组
    for(int i=1;i<=n;i++)   b[i]=a[i]-a[i-1];

    while(p--) {
        int l,r,c;  scanf("%d%d%d",&l,&r,&c);
        b[l]+=c;    b[r+1]-=c;      //修改区间的值
    }
    int ans=1e9;
    for(int i=1;i<=n;i++) {
        a[i]=a[i-1]+b[i];   //重新更新原数组
        ans=min(ans,a[i]);
    }
    cout<<ans;
    return 0;
}
```

【扩展】二维差分

如果扩展到二维，我们需要让二维数组被选中的子矩阵中的每个元素的值加上c,是否也可以达到$O(1)$的时间复杂度。答案是可以的，考虑二维差分。

`a[][]` 数组是 `b` [][]数组的前缀和数组，那么 `b` [][]是 `a` [][]的差分数组

原数组：`a[i][j]` ，我们去构造差分数组：`b[i][j]`

使得 `a` 数组中 `a[i][j]` 是 `b` 数组左上角$(1,1)$到右下角(i,j)所包围矩形元素的和。

如何构造 `b` 数组呢？

这里不像一维差分那样，可以直接找到一种构造方式去构造，因此我们可以去逆向思考。

同一维差分，我们构造二维差分数组目的是为了 让原二维数组 `a` 中所选中子矩阵中的每一个元素加上 `c` 的操作，可以由$O(n*n)$的时间复杂度优化成$O(1)$

已知原数组 `a` 中被选中的子矩阵为 以$(x1,y1)$为左上角，以$(x2,y2)$为右上角所围成的矩形区域;

始终要记得，`a` 数组是 `b` 数组的前缀和数组，比如对 `b` 数组的 `b[i]`[j]的修改，会影响到 `a` 数组中从 `a[i][j]` 及往后的每一个数。

假定我们已经构造好了 `b` 数组，类比一维差分，我们执行以下操作来使被选中的子矩阵中的每个元素的值加上 `c`

`b[x1][y1]` + = c;

`b[x1][y2+1]` - = c;

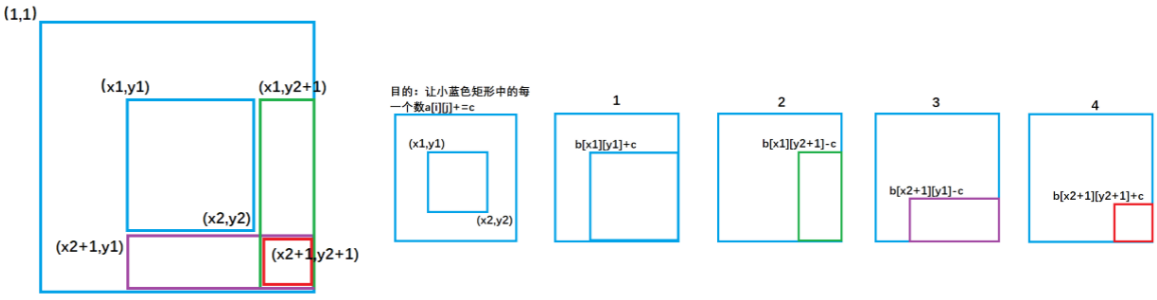
`b[x2+1][y1]` - = c;

`b[x2+1][y2+1]` + = c;

每次对 `b` 数组执行以上操作，等价于：

```
for(int i=x1;i<=x2;i++)
  for(int j=y1;j<=y2;j++)
    a[i][j]+=c;
```

我们来通过图的方式来解释一下：



`b[x1][y1] +=c`；对应图1 ,让整个 `a` 数组中蓝色矩形面积的元素都加上了 `c`。
`b[x1][y2+1]-=c`；对应图2 ,让整个 `a` 数组中绿色矩形面积的元素再减去 `c` ，使其内元素不发生改变。
`b[x2+1][y1]- =c`；对应图3 ,让整个 `a` 数组中紫色矩形面积的元素再减去 `c` ，使其内元素不发生改变。
`b[x2+1][y2+1]+=c`；对应图4,,让整个 `a` 数组中红色矩形面积的元素再加上 `c` ，红色内的相当于被减了两次，再加上一次 `c` ，才能使其恢复。

我们将上述操作封装成一个插入函数:

```
void insert(int x1,int y1,int x2,int y2,int c)
{     //对b数组执行插入操作，等价于对a数组中的(x1,y1)到(x2,y2)之间的元素都加上了c
    b[x1][y1]+=c;
    b[x2+1][y1]-=c;
    b[x1][y2+1]-=c;
    b[x2+1][y2+1]+=c;
}
```

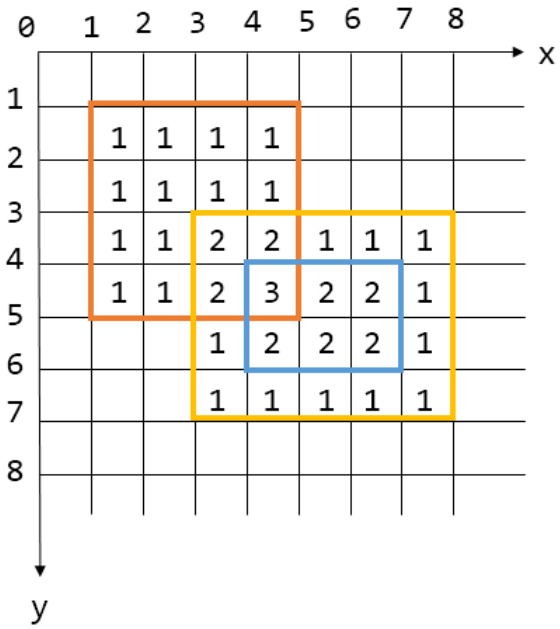
我们可以先假想`a`数组为空，那么`b`数组一开始也为空，但是实际上`a`数组并不为空，因此我们每次让以(i,j)为左上角到以(i,j)为右上角面积内元素(其实就是一个小方格的面积)去插入`c=a[i][j]`，等价于原数组`a`中`(i,j)`到`(i,j)`范围内 加上了`a[i][j]`,因此执行$n*m$次插入操作，就成功构建了差分`b`数组。

```
for(int i=1;i<=n;i++)
  {
      for(int j=1;j<=m;j++)
      {
          insert(i,j,i,j,a[i][j]);     //构建差分数组
      }
  }
```

P5542 [USACO19FEB]Painting The Barn S

算法分析：

题意的数据范围是$n <= 1000,00$。矩阵的最大为$m <= 1000$。如果我们对每一个子矩阵从(x_1,y_1)到(x_2,y_2)都标记一遍。时间复杂度是$O(n*m*m)$。很明显是会超时的。采用差分的方式，可以帮助我们将$O(m^2)$这块的时间复杂度优化到$O(1)$。从而将总的时间复杂度优化到$O(n)$。从题意中，画出坐标系和对应的子矩阵可以得到如下图形。



在代码实现上要注意的是，题目给的是坐标系的数据。

代码参考：

```
#include<bits/stdc++.h>
using namespace std;
const int N=1010;
int a[N][N],b[N][N];
int n,k;
int ans;
int main() {
    cin>>n>>k;
    for(int i=1;i<=n;i++) {
        int x1,y1,x2,y2;    scanf("%d%d%d%d",&x1,&y1,&x2,&y2);
        //坐标系：计算的时候让x1 = x1+1 ,y1 = y1+1
        b[x1+1][y1+1]++;    b[x1+1][y2+1]--;    b[x2+1][y1+1]--;    b[x2+1][y2+1]++;
    }
    //差分转前缀和
    for(int i=1;i<=1000;i++) {
        for(int j=1;j<=1000;j++) {
            a[i][j]=a[i-1][j]+a[i][j-1]-a[i-1][j-1]+b[i][j];
            if(a[i][j]==k)  ans++;
        }
    }
    cout<<ans;
    return 0;
}
```

例题推荐

- P4552 [Poetize6] IncDec Sequence
- P3948 数据结构
- P3655 不成熟的梦想家 (未熟DREAMER)

扩展：

感兴趣的同学可以提前了解下：树的最近公共祖先问题，树上前缀和。树上差分。这块内容是提高组非常喜欢考查的知识点。

倍增

倍增就是“成倍增长”，每次递增2倍。
倍增法的题目都利用了数的二进制表示（二进制划分）：
$N = a_0 2^0 + a_1 2^1 + a_2 2^2 + a_3 2^3 + a_4 2^4 + \ldots$。

例如：35，它的二进制是100011，第5,1,0位是1。即$a_5 = a_1 = a_0 = 1$，把这几位的权值相加，有$35 = 2^5 + 2^1 + 2^0 = 32 + 2 + 1$。数的二进制划分反映了一种快速增长的特性。第$i$位的权值$2^i$等于前面所有权值的和加 1：$2^i = 2^{i-1} + 2^{i-2} + 2^{i-3} + \ldots + 2^0 + 1$。

一个整数n，它的二进制表示只有$logn$ 位。如果要从0增长到n，可以用$1, 2, 4, \ldots, 2^k$为“跳板“，快速跳到n，这些”跳板“只有$k = logn$个。

倍增法的特点是需要提前计算出第$1, 2, 4, \ldots, 2^k$个跳板，这要求数据是静态不变的，不是动态变化的。如果数据发生了变化，所有跳板要重新计算，跳板就失去了意义。

倍增法的经典应用有：矩阵快速幂、ST算法，LCA（最近公共祖先）、后缀数组。

ST(Sparse Table) 表

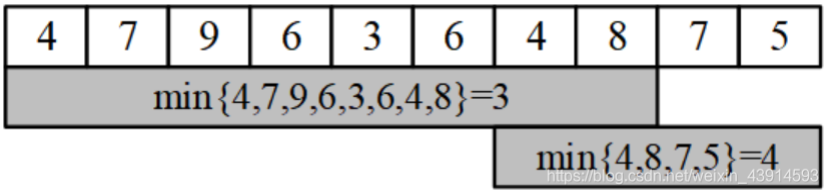
ST算法是求解RMQ问题的优秀算法，它适用于静态空间的RMQ查询。

静态空间的RMQ问题（Range Minimum Query，区间最大最小值问题）：给定长度为n的静态数列，做m次询问，每次给定$L, R \leq n$，查询区间$[L, R]$内的最值。

下面都以最小值为例。

用暴力搜区间$[L, R]$的最小值，即逐一比较区间内的每个数，复杂度是$O(n)$的；m次查询，复杂度$O(mn)$。暴力法的效率很低。

ST算法源于这样一个原理：一个大区间若能被两个小区间覆盖，那么大区间的最值等于两个小区间的最值。例如下图中，大区间$\{4,7,9,6,3,6,4,8,7,5\}$被两个小区间$\{4,7,9,6,3,6,4,8\}$、$\{4,8,7,5\}$覆盖，大区间的最小值3，大区间的最小值4，整个区间的最小值等于两个小区间的最小值，$min\{3,4\}=3$。这个例子特意让两个小区间有部分重合，因为重合不影响结果。



ST表主要有两个步骤：

1. 预处理区间的最小值。总共最多分出$log\ n$组。所以总的时间复杂度为$O(nlog\ n)$。

	原数列	4	7	9	6	3	6	4	8	7	5
		ST 预处理表（区间倍增） f[i,j]表示：以i为起点，长度为2^j 的最值									
0	区间大小	[1,1]	[2,2]	[3,3]	[4,4]	[5,5]	[6,6]	[7,7]	[8,8]	[9,9]	[10,10]
	表示f[i,j]	f[1,0]	f[2,0]	f[3,0]	f[4,0]	f[5,0]	f[6,0]	f[7,0]	f[8,0]	f[9,0]	f[10,0]
		4	7	9	6	3	6	4	8	7	5
1	区间大小	[1,2]	[2,3]	[3,4]	[4,5]	[5,6]	[6,7]	[7,8]	[8,9]	[9,10]	
	表示f[i,j]	f[1,1]	f[2,1]	f[3,1]	f[4,1]	f[5,1]	f[6,1]	f[7,1]	f[8,1]	f[9,1]	
		4	7	6	3	3	4	4	7	5	
2	区间大小	[1,4]	[2,5]	[3,6]	[4,7]	[5,8]	[6,9]	[7,10]			
	表示f[i,j]	f[1,2]	f[2,2]	f[3,2]	f[4,2]	f[5,2]	f[6,2]	f[7,2]			
		4	3	3	3	3	4	4			
3	区间大小	[1,8]	[2,9]	[3,10]							
	表示f[i,j]	f[1,3]	f[2,3]	f[3,3]							
		3	3	3							
	区间大小	[1,10]									
	表示f[i,j]	f[1,4]									
		3									

```
void ST_work() {
        //区间倍增预处理。
    for(int i=1;i<=n;i++)   f[i][0] = a[i]; //初始化
    int t = log2(n) +1; //以2为底
    for(int j=1;j<t;j++) {
        for(int i=1;i<=n-(1<<j)+1;i++) {
            f[i][j] = min(f[i][j-1],f[i+(1<<(j-1))][j-1]);
            cout<<i<<" "<<pow(2,j)<<" "<<f[i][j]<<endl;  //可以打印查找预处理的结果
        }
    }
}
```

2. 查询区间最小值。每次查询的时间复杂度为$O(1)$。

当询问任意区间$[l,r]$的最值时，我们先计算出一个k，满足$2^k < (r-l+1) \leq 2^{k+1}$,也就是使2的$k$次幂小于区间长度的前提下最大的$k$。那么"从 l 开始的 2^k个数"和"以 r 结尾的2^k个数"这两段一定覆盖了整个区间$[l,r]$，这两段的最小值分别是$f[l,k]$和$f[r-2^k+1][k]$，二者中较大的那个就是整个区间$[l,r]$的最值。因为求得是最小值，所以这两段只要覆盖区间$[l,r]$即可，即使有重叠也没关系（如下图）。

ST 表区间最值查询

查询区间	[1,6]	[2,7]	[4,8]	[1,9]
k值 $\log_2$（R-L+1）	2	2	2	3
分解区间	f[1,2]	f[2,2]	f[4,2]	f[1,3]
	f[2,2]	f[3,2]	f[5,2]	f[2,3]
	f[3,2]	f[4,2]		
最值	3	3	3	3

```
//查询区间最值
int ST_query(int L,int R) {
    int k = log(R-L+1) / log(2);
    return max(f[L][k],f[R-(1<<k)+1][k]);
}
```

P3865 【模板】ST表

算法分析：

从题目数据上分析：M次循环是不能优化。如果直接暴力枚举每一个区间的最大值。时间复杂度是$O(n)$。M次循环，总的时间复杂度就是$O(MN)$。而$1 \leq N \leq 10^5, 1 \leq M \leq 2 \times 10^6$。这样会超时$TLE$。 采用$ST$表的方法。预处理的时间复杂度是$O(nlog\,n)$。 查询区间最值的时间复杂度是$O(1)$。所以总的时间复杂度是$O(nlog\,n+m)$。

代码参考：

```
#include<iostream>
#include<cstdio>
#include<cmath>
using namespace std;
const int N=1000010;
int a[N],f[N][20];
```

```cpp
int n,m;
void ST_work() {
    for(int i=1;i<=n;i++)   f[i][0] = a[i];
    int t = log2(n) +1; //以2为底
    for(int j=1;j<t;j++) {
        for(int i=1;i<=n-(1<<j)+1;i++) {
            f[i][j] = max(f[i][j-1],f[i+(1<<(j-1))][j-1]);
        }
    }
}
int ST_query(int L,int R) {
    int k = log(R-L+1) / log(2);
    return max(f[L][k],f[R-(1<<k)+1][k]);
}
int main() {

    cin>>n>>m;
    for(int i=1;i<=n;i++)   scanf("%d",&a[i]);
    ST_work();  //倍增预处理
    for(int i=1;i<=m;i++) {
        int l,r;
        scanf("%d %d",&l,&r);
        printf("%d \n",ST_query(l,r));
    }
    return 0;
}
```

总结：

ST算法与线段树

求解RMQ问题更常用的是线段树，它求RMQ的时间复杂度与ST差不多，但是两者有很大区别：线段树用于动态数组，ST用于静态数组。

ST算法用$O(nlogn)$时间来预处理数组，预处理之后每次区间查询是$O(1)$的。如果数组是动态改变的，改变一次就需要用$O(nlogn)$预处理一次，导致效率很低。线段树适用于动态维护（添加或删除）的数组，它每次维护数组和每次查询都是$O(logn)$的。从数组是否能动态维护这个角度来说，ST是离线算法，线段树是在线算法。ST的优势是编码简单，如果数组是静态的，就用ST。

ST算法的适用场合：

从ST算法的原理看，它的核心思想是“大区间被两个小区间覆盖、小区间的重复覆盖不影响结果”，然后用倍增法划分小区间和计算小区间的最值。最大值和最小值符合这种场景，类似的有RGQ问题（Range GCD Query，区间最大公约数问题）：查询给定区间的GCD。

例题推荐

- P1816 忠诚
- P2251 质量检测
- P2880 [USACO07JAN]Balanced Lineup G

挑战推荐：

- P2048 [NOI2010] 超级钢琴
- P4155 [SCOI2015]国旗计划
- P5465 [PKUSC2018]星际穿越
- P3295 [SCOI2016]萌萌哒